Commonware 심층 분석: Anti-Framework 블록체인 프리미티브 라이브러리

1. 개요

1.1 Commonware란?

Commonware는 Rust 기반의 오픈소스 블록체인 프리미티브 라이브러리입니다. 기존의 Cosmos SDK, OP Stack 같은 "프레임워크"와 달리, Commonware는 **“Anti-Framework”**를 표방합니다. 개발자가 필요한 컴포넌트만 선택적으로 조합하여 커스텀 블록체인을 구축할 수 있도록 설계되었습니다.

> “Today’s blockchain stack is designed to be a jack of all trades, but master of none. Modeled after virtual machines (VM) and kernels, it exposes a balanced interface through a one-size-fits-all framework instead of molding to the needs of any specific application.”
> — Commonware

1.2 창립 배경

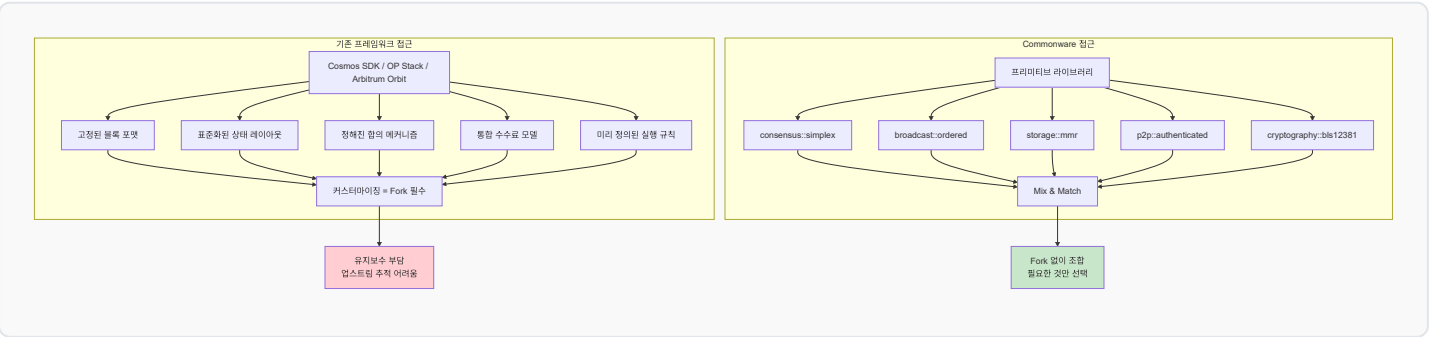
창립자: Patrick O’Grady

- Ava Labs VP of Engineering 출신
- Coinbase에서 Rosetta (현 Coinbase Mesh API) 개발
- Avalanche HyperSDK 아키텍트

펀딩:

- Seed Round: $9M (Haun Ventures, Dragonfly Capital 공동 리드)
- Strategic Investment: $25M (Tempo 리드, 2025년 11월)
- 엔젤 투자자: Berachain Smokey the Bera, Cosmos Zaki Manian, EigenLayer Sreeram Kannan, Farcaster Dan Romero, Helius Mert Mumtaz

1.3 핵심 철학: Anti-Framework



기존 프레임워크의 문제점:

프로젝트	사용 프레임워크	Fork 여부	문제점
Sei	CometBFT	Fork 필수	합의 커스터마이징
Celestia	CometBFT	Fork 필수	DA 레이어 특화
dYdX	CometBFT	Fork 필수	오더북 최적화
Penumbra	CometBFT	Fork 필수	프라이버시 기능
수많은 L2	OP Stack	Fork 필수	실행 규칙 변경

> "Frameworks today like the Cosmos SDK and CometBFT is open and flexible in a few specific ways, but I argue that you get stuck eventually. You have to redo and fork and rebuild a lot of stuff, and builders end up spending too much time dealing with the abstractions rather than focusing on the product."
> — Patrick O'Grady

---

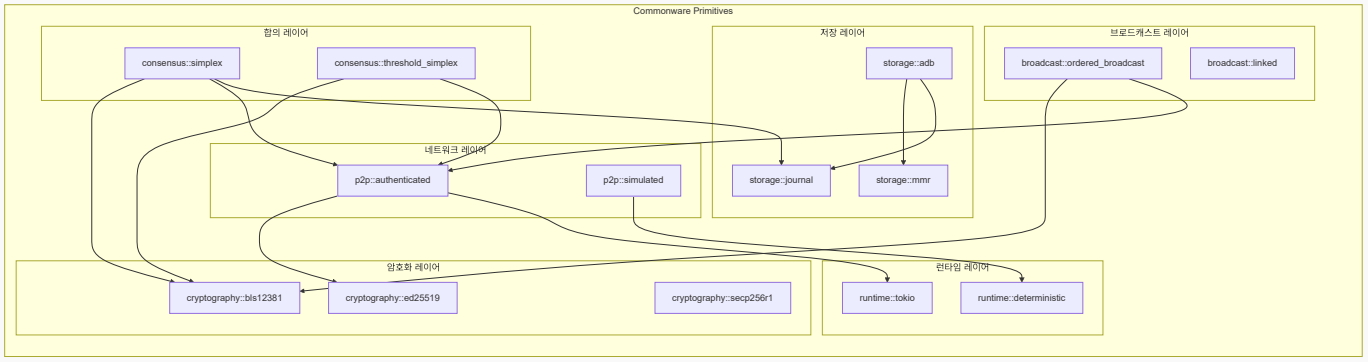
2. Commonware Library 구조

2.1 전체 아키텍처

```
commonware/
├── primitives/             # 핵심 프리미티브
|    ├── broadcast/          # 데이터 전파
|    ├── consensus/          # 합의 알고리즘
|    ├── cryptography/       # 암호화
|    ├── p2p/                # P2P 네트워킹
|    ├── runtime/            # 런타임 환경
|    └── storage/            # 저장소
|
├── examples/               # 예제 구현
|    ├── alto/               # 미니멀 블록체인
|    ├── battleware/         # VRF 기반 온체인 게임
|    ├── bridge/             # 크로스체인 브릿지
|    ├── chat/               # 암호화 채팅
|    ├── estimator/          # 메커니즘 시뮬레이터
|    ├── flood/              # P2P 스트레스 테스트
|    ├── log/                # 비밀 로그 합의
|    ├── reshare/            # 임계 비밀 리셰어링
|    └── sync/               # 상태 동기화
|
└── tools/                  # 도구
     ├── docs/              # 문서
     ├── docker/            # 컨테이너
```

```
    ├── macros/              # 프로시저 매크로
    └── mcp/                 # MCP 서버
```

2.2 프리미티브 의존성 그래프



2.3 Commonware vs Cosmos SDK 상세 비교

구분	Cosmos SDK	Commonware
언어	Go	Rust
철학	완전한 프레임워크	프리미티브 라이브러리
합의	CometBFT (필수)	Simplex (선택적)
모듈 시스템	x/bank, x/staking, x/auth...	없음 (개발자 정의)
블록 포맷	Protobuf 정의됨	없음 (개발자 정의)
상태 레이아웃	IAVL Tree 고정	없음 (개발자 정의)
Finality 정의	고정 (1블록)	없음 (개발자 정의)
Mempool 정책	고정	없음 (개발자 정의)
수수료 모델	gas × gasPrice	없음 (개발자 정의)
IBC 지원	네이티브	별도 구현 필요
학습 곡선	프레임워크 전체 이해	필요한 프리미티브만
커스터마이징	Fork 또는 모듈 확장	Mix & Match
라이선스	Apache 2.0	Apache 2.0 + MIT

---

3. 합의 프리미티브: Simplex Consensus

3.1 Simplex 배경

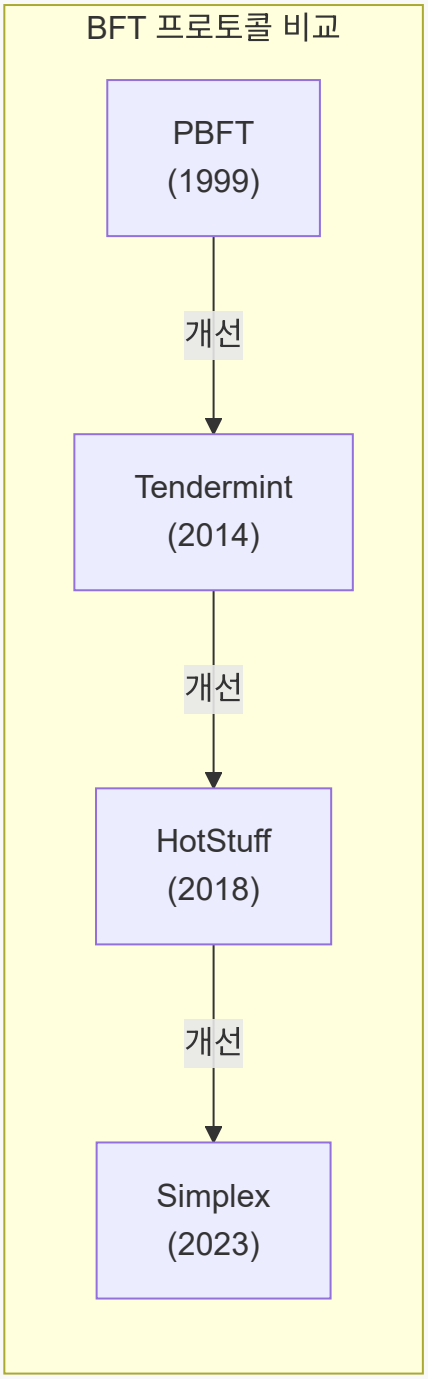
Simplex Consensus는 **Benjamin Chan**과 **Rafael Pass**가 2023년 TCC(Theory of Cryptography Conference)에서 발표한 BFT 프로토콜입니다.

> "Simplex is a new consensus protocol that is faster than the state-of-the-art, and even easier to understand. It has both Byzantine and Crash-fault variants. It is suitable for both teaching and for implementation in practice."
> — Simplex Consensus

현재 채택 현황:

- **Commonware Library**: 최초의 프로덕션 구현 (Rust)
- **Solana Alpenglow**: 차세대 합의로 채택 예정 (수정 버전)
- **Tempo**: 결제 특화 L1에서 사용

3.2 Simplex vs 기존 BFT 프로토콜



구분	PBFT	Tendermint	HotStuff	Simplex
라운드 수	3	3	3	2-3
View Change Timeout	복잡	6Δ	가변	**3Δ**
리더 대기 시간	2Δ	2Δ	2Δ	**없음**
메시지 복잡도	$O(n^2)$	O(n)	O(n)	O(n)
Finality 조건	복잡	복잡	복잡	**단순**
구현 난이도	높음	중간	중간	**낮음**

3.3 Simplex 프로토콜 상세

3.3.1 핵심 혁신

1. 즉시 View 전환 (Change #1)

기존 프로토콜에서는 Finalize 메시지를 보낸 후에도 해당 view에 머물러 View-change를 보낼 수 있었습니다. Simplex에서는 **Cert(k, x)를 수집하면 즉시 다음 view로 이동**합니다.

```
 기존 (Tendermint):
  Finalize(k, x) 전송 → view k에 머무름 → View-change(k) 가능

Simplex:
  Finalize(k, x) 전송 → 즉시 view k+1로 이동 → View-change(k) 불가
```

2. No-Commit 증명 (Change #2)

n-f개의 `View-change(k)` 메시지는 **해당 view에서 결정이 없었음을 증명**합니다.

```
 정리: n-f View-change(k)와 n-f Finalize(k, *) 는 상호 배타적
증명: 정직한 파티가 Finalize를 보내면 즉시 다음 view로 이동하므로
     View-change(k)를 절대 보내지 않음
```

이 속성 덕분에 리더는 자신의 인증서보다 높은 view들에 대해 n-f개의 View-change가 있으면, 더 높은 인증서가 없음을 확신할 수 있습니다.

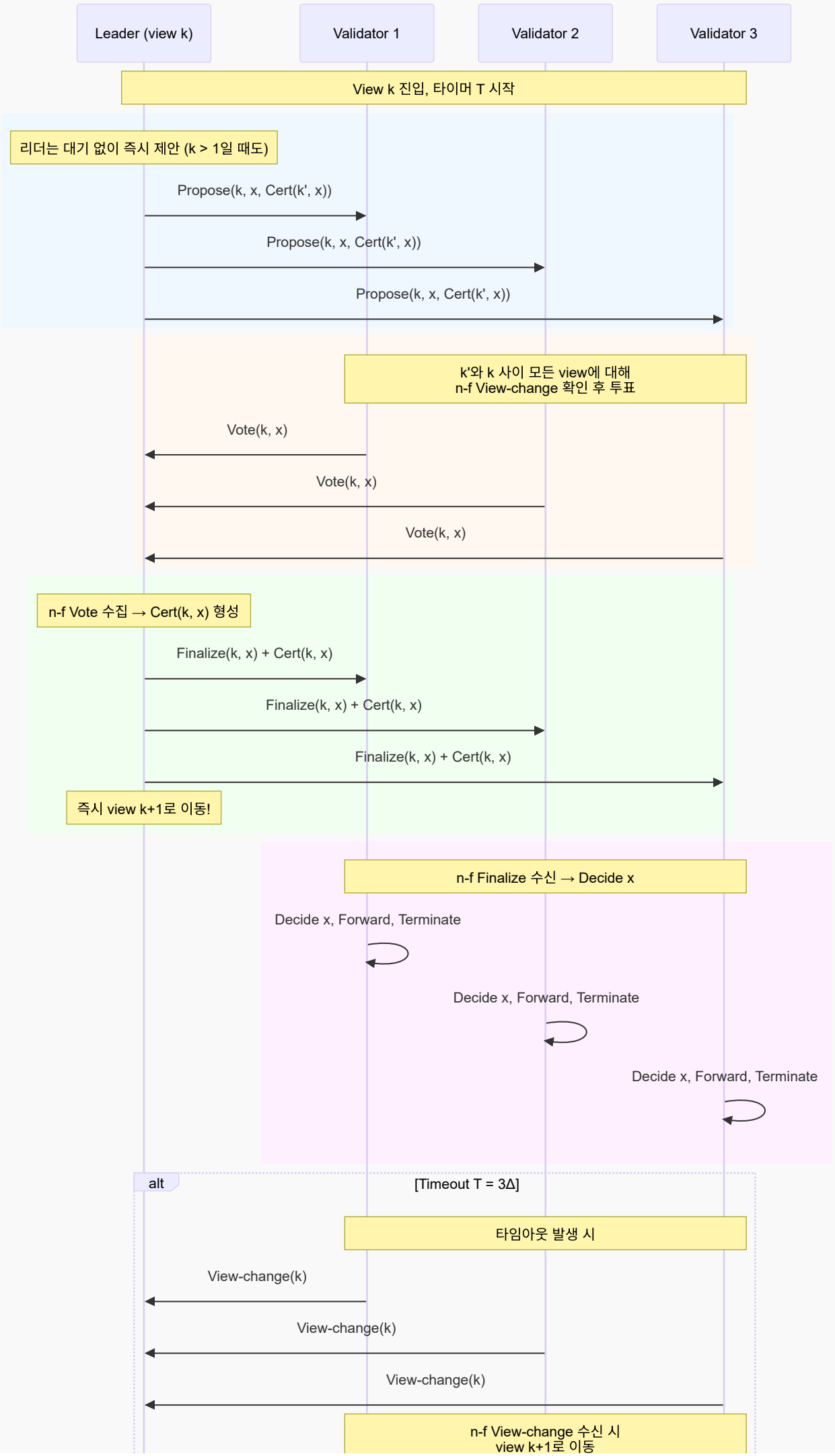
3. 리더 대기 제거 (Change #3)

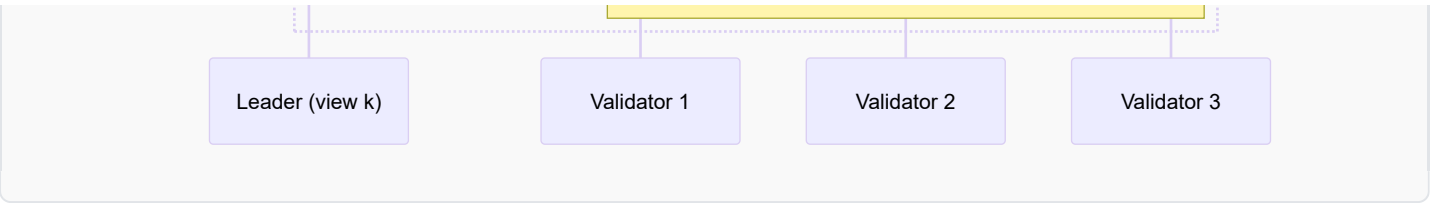
기존 프로토콜에서 리더는 2Δ 동안 대기한 후 제안했습니다. Simplex에서는 이 대기가 **완전히 제거**됩니다.

4. 짧은 Timeout (Change #4)

View-change timeout이 **6Δ에서 3Δ로 단축**됩니다.

3.3.2 프로토콜 흐름


Leader (view k)
Validator 1
Validator 2
Validator 3
View k 진입, 타이머 T 시작
리더는 대기 없이 즉시 제안 (k > 1일 때도)
Propose(k, x, Cert(k', x))
Propose(k, x, Cert(k', x))
Propose(k, x, Cert(k', x))
k'와 k 사이 모든 view에 대해
n-f View-change 확인 후 투표
Vote(k, x)
Vote(k, x)
Vote(k, x)
n-f Vote 수집 → Cert(k, x) 형성
Finalize(k, x) + Cert(k, x)
Finalize(k, x) + Cert(k, x)
Finalize(k, x) + Cert(k, x)
즉시 view k+1로 이동!
n-f Finalize 수신 → Decide x
Decide x, Forward, Terminate
Decide x, Forward, Terminate
Decide x, Forward, Terminate
alt
[Timeout T = 3Δ]
타임아웃 발생 시
View-change(k)
View-change(k)
View-change(k)
n-f View-change 수신 시
view k+1로 이동



3.3.3 Simplex 의사 코드

```
Upon entering view k:
  Everyone starts a local timer T
  // Change #3: 리더는 k > 1일 때도 대기 없이 제안
  Leader sends (Propose, k, x, Cert(k', x)) for the highest k'

Upon receiving the first (Propose, k, x, Cert(k', x)) in view k:
  // Change #2: 더 높은 인증서 대신 View-change 증명 확인
  Send (Vote, k, x) if having seen n-f (View-change, l) for all k' < l < k

Upon receiving n-f (Vote, k, x) in view k:  // Cert(k, x) 형성
  Send (Finalize, k, x)
  Forward Cert(k, x)
  Enter view k+1  // Change #1: 즉시 다음 view로!

Upon receiving n-f (Finalize, k, x):
  Decide x
  Forward these n-f (Finalize, k, x)
  Terminate

Upon T = 3Δ  in view k:  // Change #4: 6Δ → 3Δ
  Send (View-change, k)
  Exit view k

Upon receiving n-f (View-change, k):
  Forward these n-f (View-change, k)
  Enter view k+1
```

3.4 consensus::simplex 구현

Commonware의 `consensus::simplex` 는 Simplex 프로토콜의 프로덕션 구현입니다.

3.4.1 주요 특징

```
// Commonware Simplex 설정 예시 (개념적)
use commonware_consensus::simplex::{Engine, Config};
use commonware_cryptography::{bls12381, ed25519, secp256r1};

// 다양한 서명 스킴 지원
let config = Config {
    participants: validators,
    leader: leader::Rotation::default(),
```

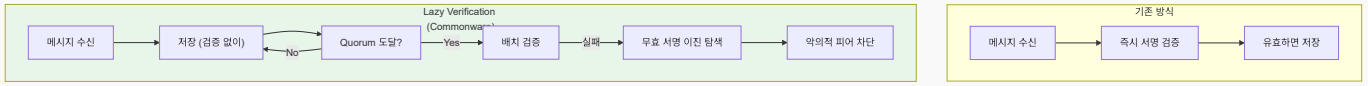
```
    // 서명 스킴 선택
    scheme: match preference {
        "ed25519" => scheme::ed25519::new(),      // 빠른 검증
        "bls_multisig" => scheme::bls12381_multisig::new(),  // 집계 가능
        "bls_threshold" => scheme::bls12381_threshold::new(), // 임계 서명
        "secp256r1" => scheme::secp256r1::new(),  // HSM 호환
    },
};
```

3.4.2 지원 서명 스킴

스킴	특징	인증서 크기	배치 검증	사용 사례
ed25519	빠른 서명/검증	O(n)	지원	일반 용도
bls12381_multisig	서명 집계 가능	O(n)	지원	많은 검증자
bls12381_threshold	임계 서명	**O(1)** (96 bytes)	지원	크로스체인, VRF
secp256r1	HSM 호환 (NIST P-256)	O(n)	미지원	엔터프라이즈

3.4.3 Lazy Verification

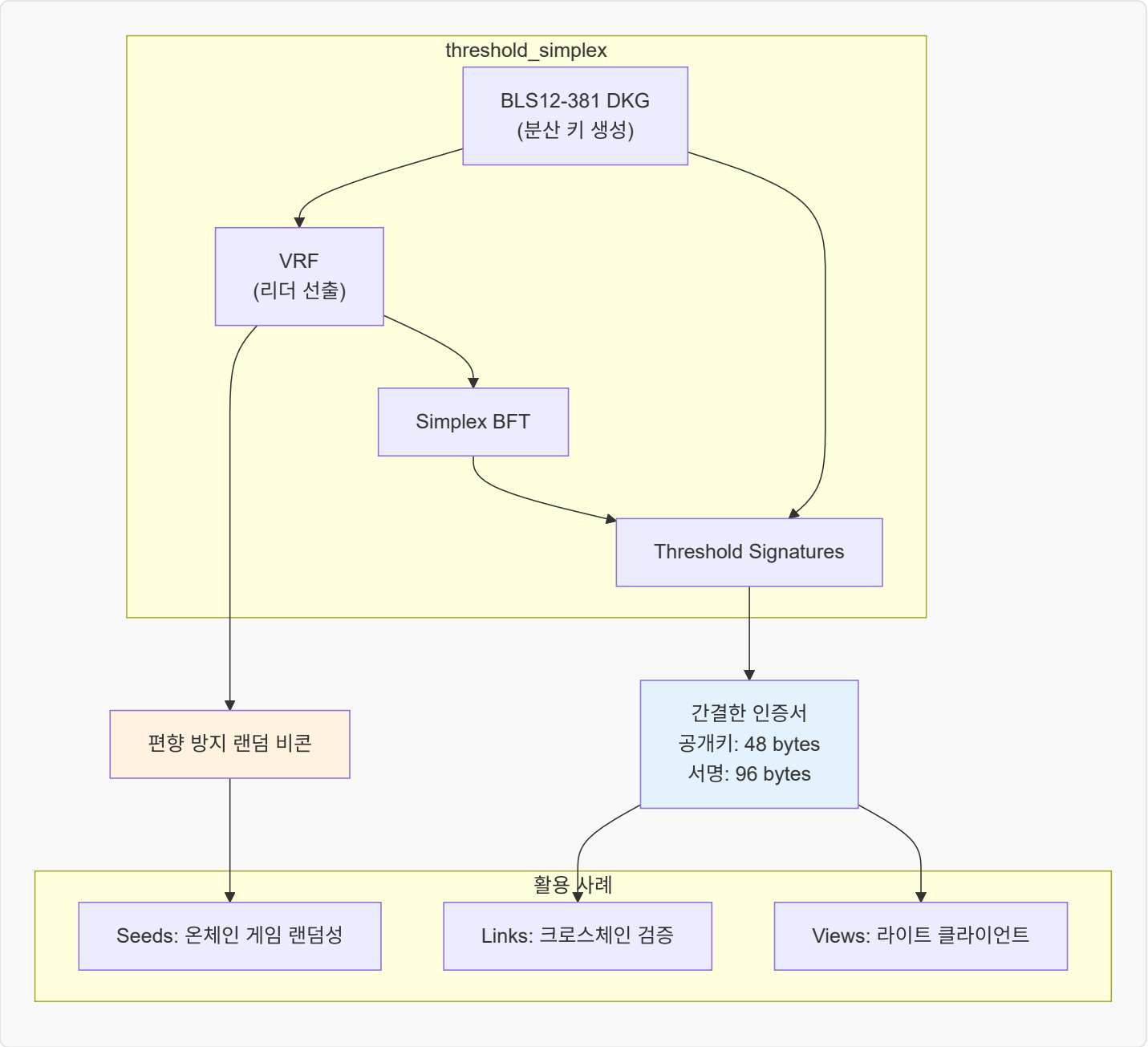
Commonware의 핵심 최적화 중 하나입니다:



> "Unlike other consensus constructions that verify all incoming messages received from peers, for schemes where `Scheme::is_batchable()` returns true, simplex lazily verifies messages (only when a quorum is met), enabling efficient batch verification."

3.5 consensus::threshold_simplex

Simplex에 **VRF**와 **간결한 합의 인증서**를 추가한 변형입니다.



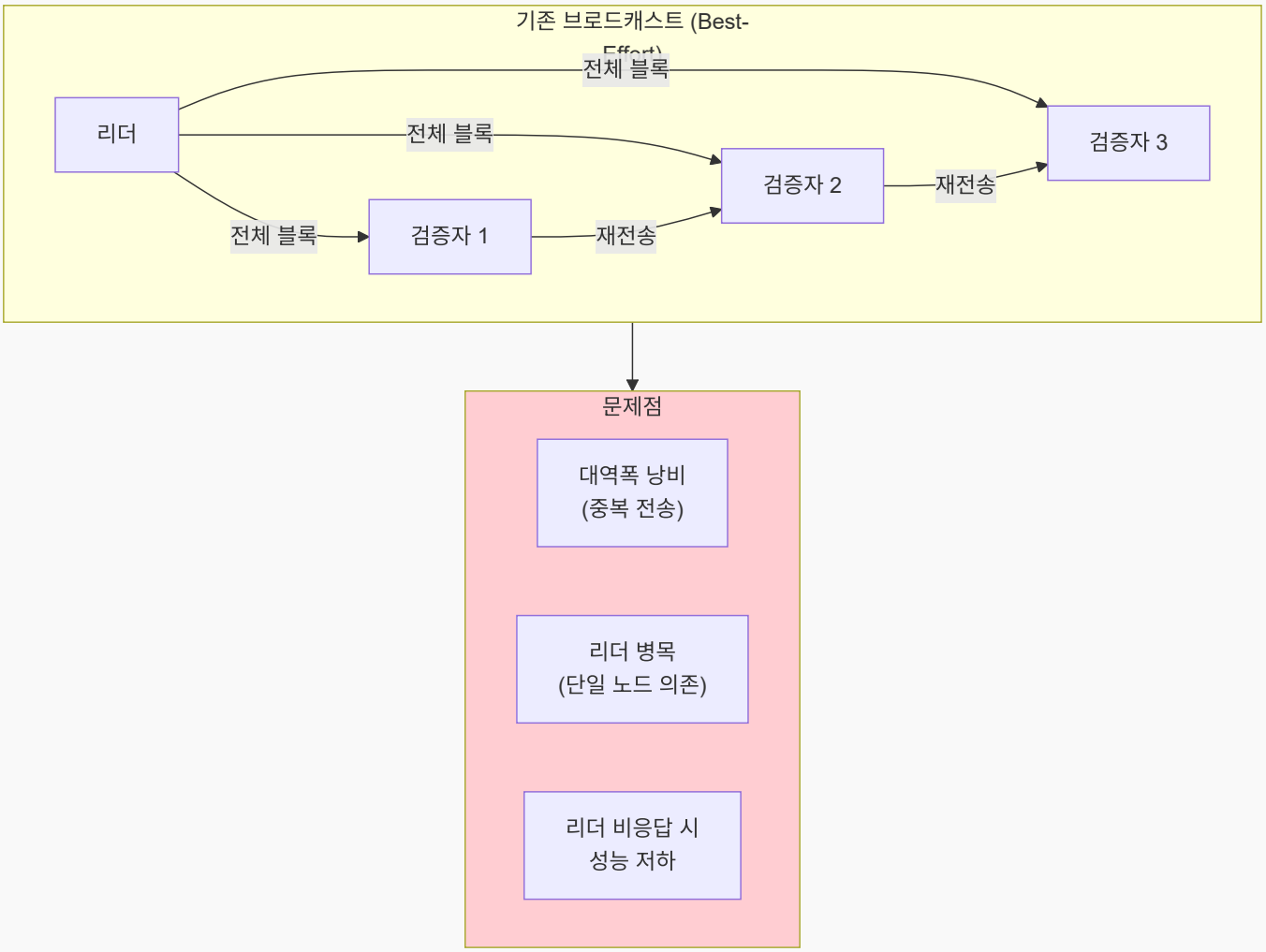
핵심 장점:

1. **간결한 인증서**: 정적 공개키(48 bytes) + 서명(96 bytes)만으로 어떤 블록이든 검증 가능
2. **크로스체인 검증**: 다른 체인에서 라이트 클라이언트 없이 상태 검증
3. **편향 방지 랜덤성**: 온체인 게임, 공정한 리더 선출에 활용
4. **제로 메시지 오버헤드**: 기존 합의 메시지에 자연스럽게 통합

---

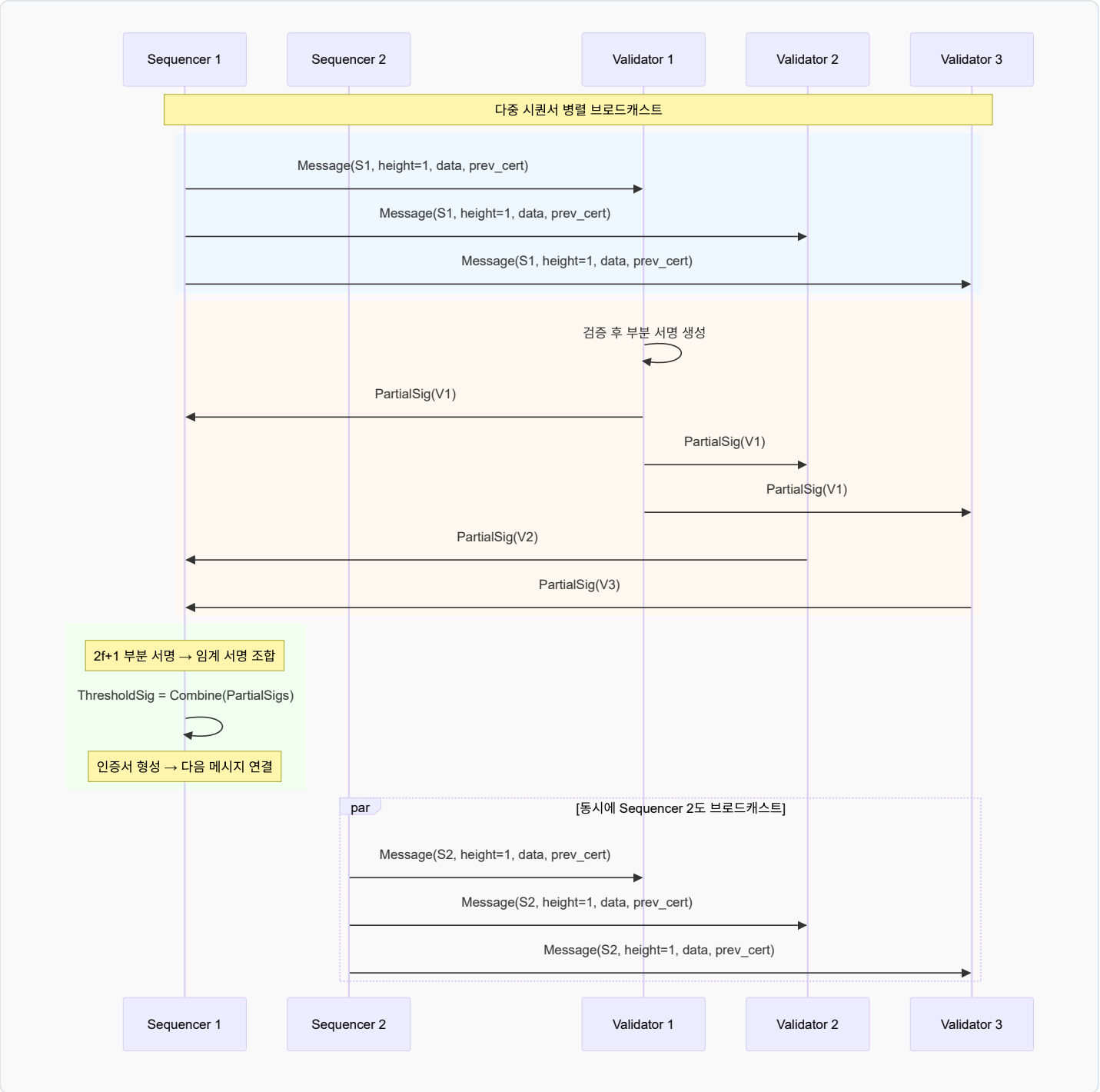
4. 브로드캐스트 프리미티브

4.1 기존 블록체인 데이터 전파의 문제점



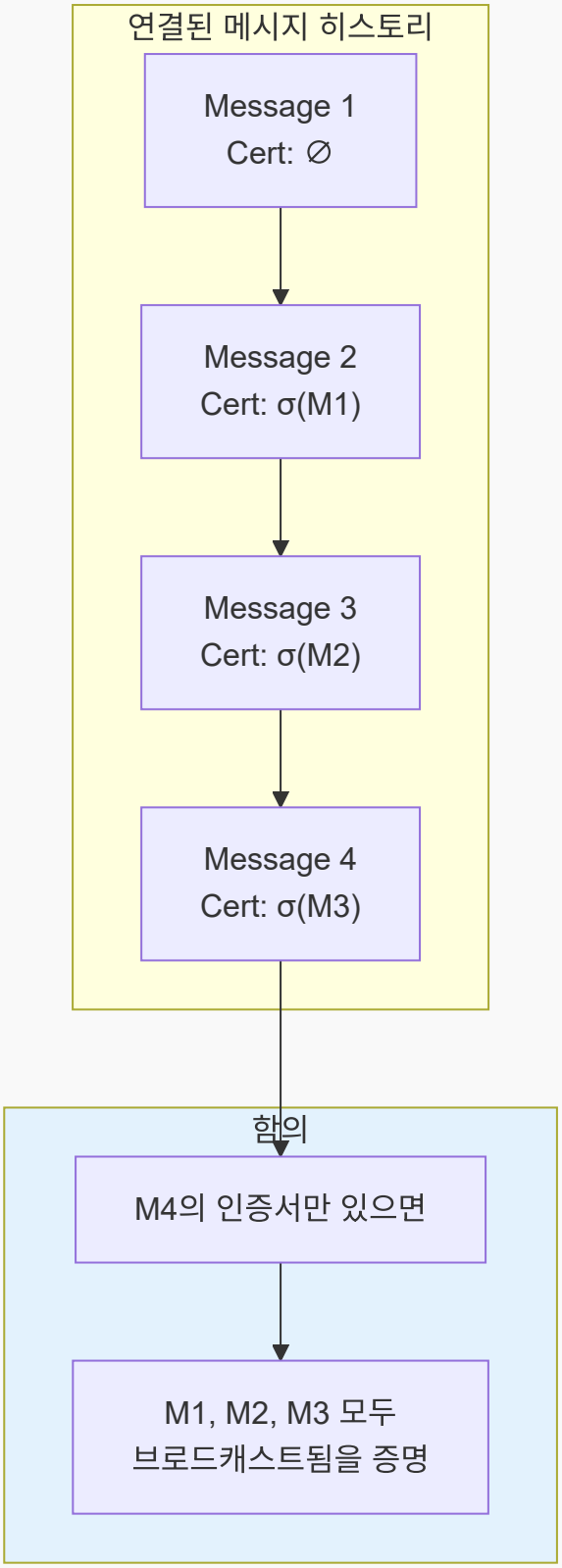
4.2 broadcast::ordered_broadcast

Commonware의 데이터 전파 프리미티브입니다.



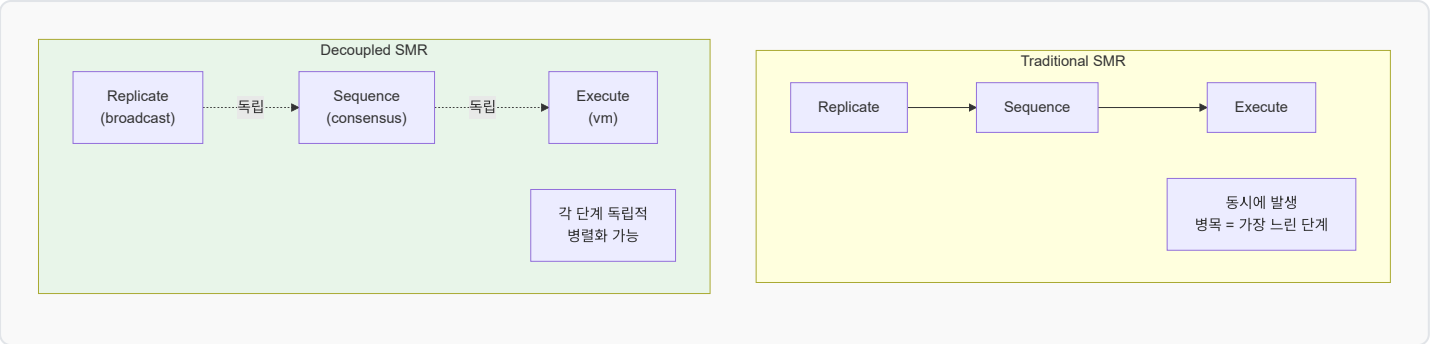
핵심 특징:

1. **합의와 분리**: 브로드캐스트는 합의 없이 독립적으로 동작
2. **다중 시퀀서**: 리더 병목 없이 병렬 데이터 전파
3. **연결된 인증서**: 각 메시지가 이전 메시지의 인증서를 포함
4. **Pre-Consensus Receipt**: 임계 서명 = 포함 보장 증명



4.3 DSMR (Decoupled State Machine Replication)

Commonware는 **DSMR** 아키텍처를 지원합니다.

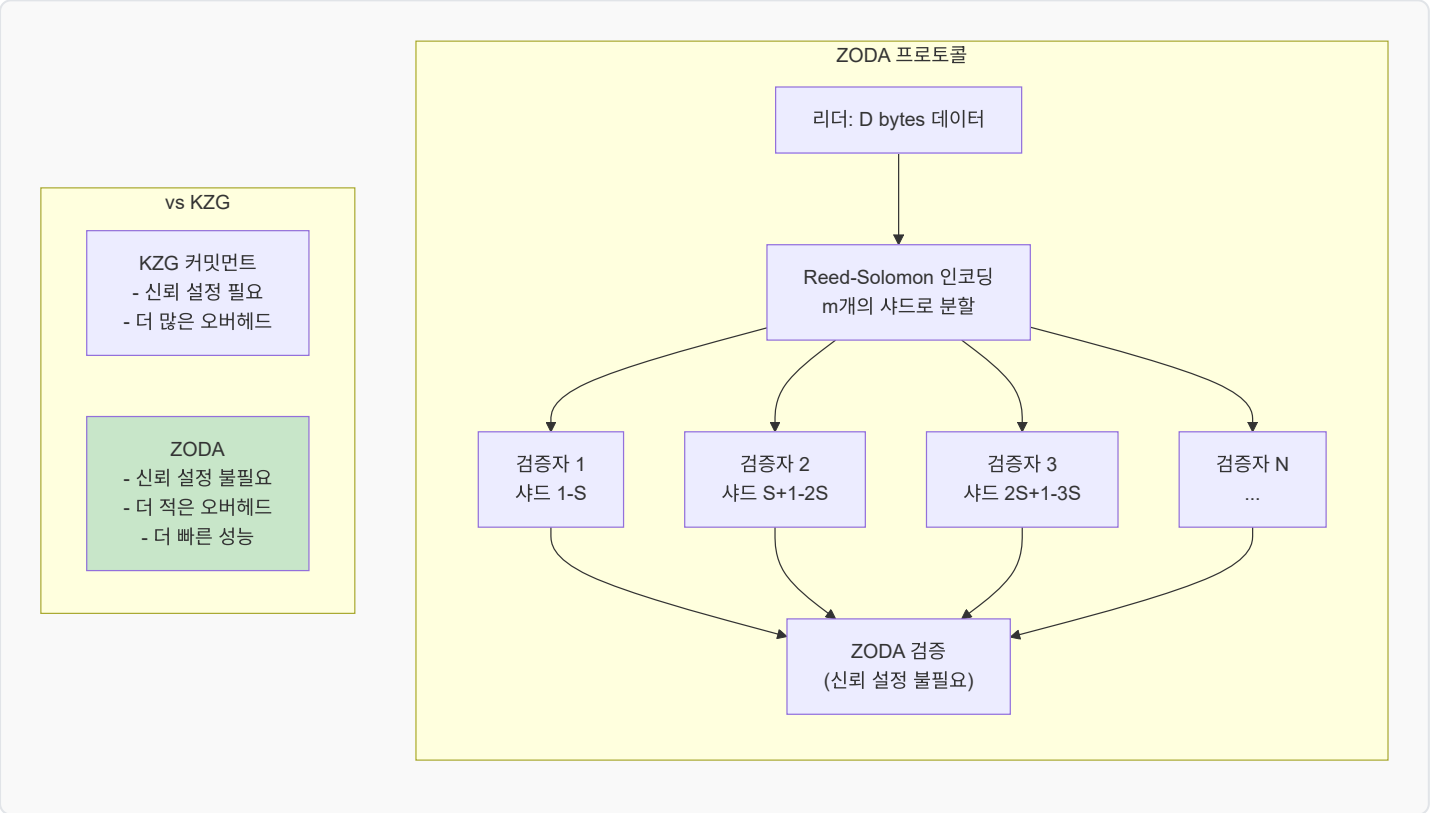


DSMR 장점:

- 브로드캐스트를 합의와 분리하여 처리량 증가
- 합의는 각 시퀀서의 tip만 포함하면 됨 (소량의 데이터)
- 리더가 느려도 다른 시퀀서가 데이터 전파 가능

4.4 ZODA (Zero-Overhead Data Availability)

대용량 블록의 효율적 전파를 위한 최신 프리미티브입니다.

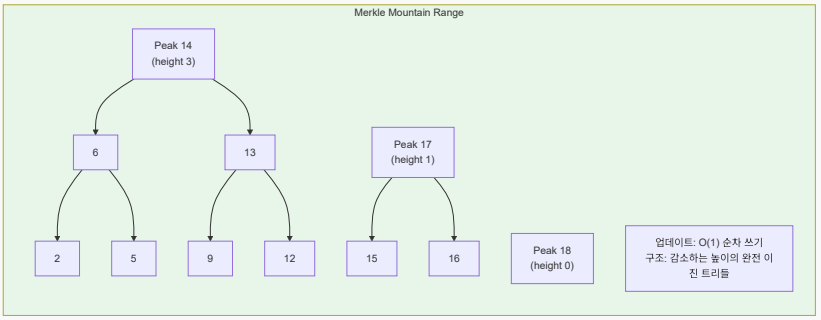


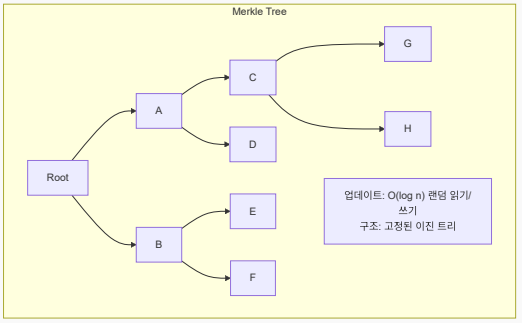
---

5. 저장 프리미티브

5.1 storage::mmr (Merkle Mountain Range)

5.1.1 MMR vs Merkle Tree





MMR 특징:

구분	Merkle Tree	MMR
구조	단일 완전 이진 트리	감소하는 높이의 트리 리스트
업데이트	O(log n) 랜덤 I/O	**O(1) 순차 쓰기**
수정	임의 위치 가능	**Append-only**
삭제	가능	불가 (논리적 삭제만)
SSD 친화성	낮음 (랜덤 쓰기)	**높음 (순차 쓰기)**
GC/Compaction	필요	**불필요**

5.1.2 MMR 구조

```
 11개 원소 추가 후 MMR (19개 노드):

Height
  3              14
               /   ₩
              /     ₩
             /       ₩
            /         ₩
  2        6           13
         / ₩         /  ₩
  1      2   5       9    12    17
        / ₩ / ₩     / ₩  / ₩   / ₩
  0    0  1 3  4   7  8 10 11 15 16  18

Position 0  1  2  3  4  5  6  7  8  9 10

Peaks: [14, 17, 18]
Root = Hash(19, Hash(14), Hash(17), Hash(18))  // "Bagging the peaks"
```

5.1.3 MMR 증명

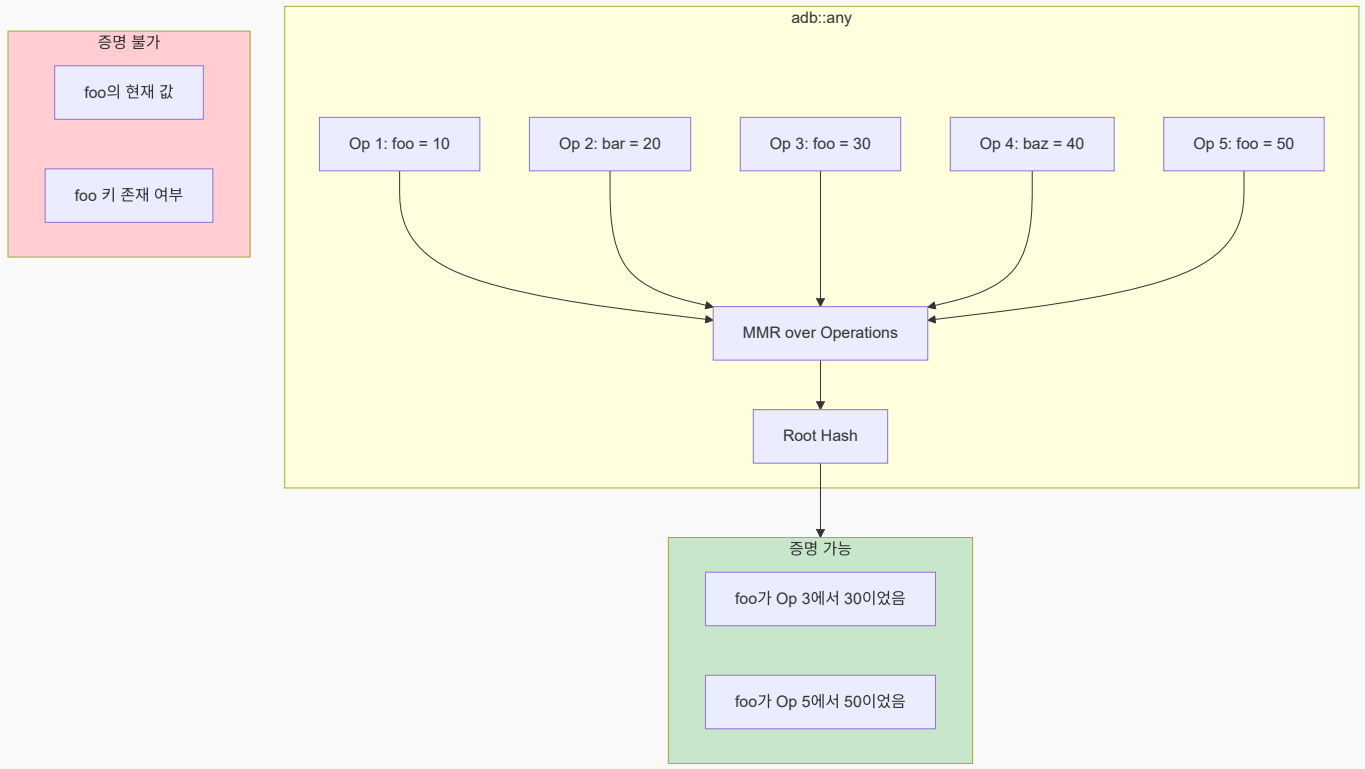
```
// MMR 포함 증명 예시
struct MMRProof {
    // 리프에서 피크까지의 경로
    path_to_peak: Vec<Hash>,
    // 다른 피크들의 해시
    other_peaks: Vec<Hash>,
    // 전체 MMR 크기
    mmr_size: u64,
}

// 증명 크기: O(log n) - Merkle Tree와 동일
// 검증: 경로 해시 → 피크 → 루트 (bagging)
```

5.2 storage::adb (Authenticated Database)

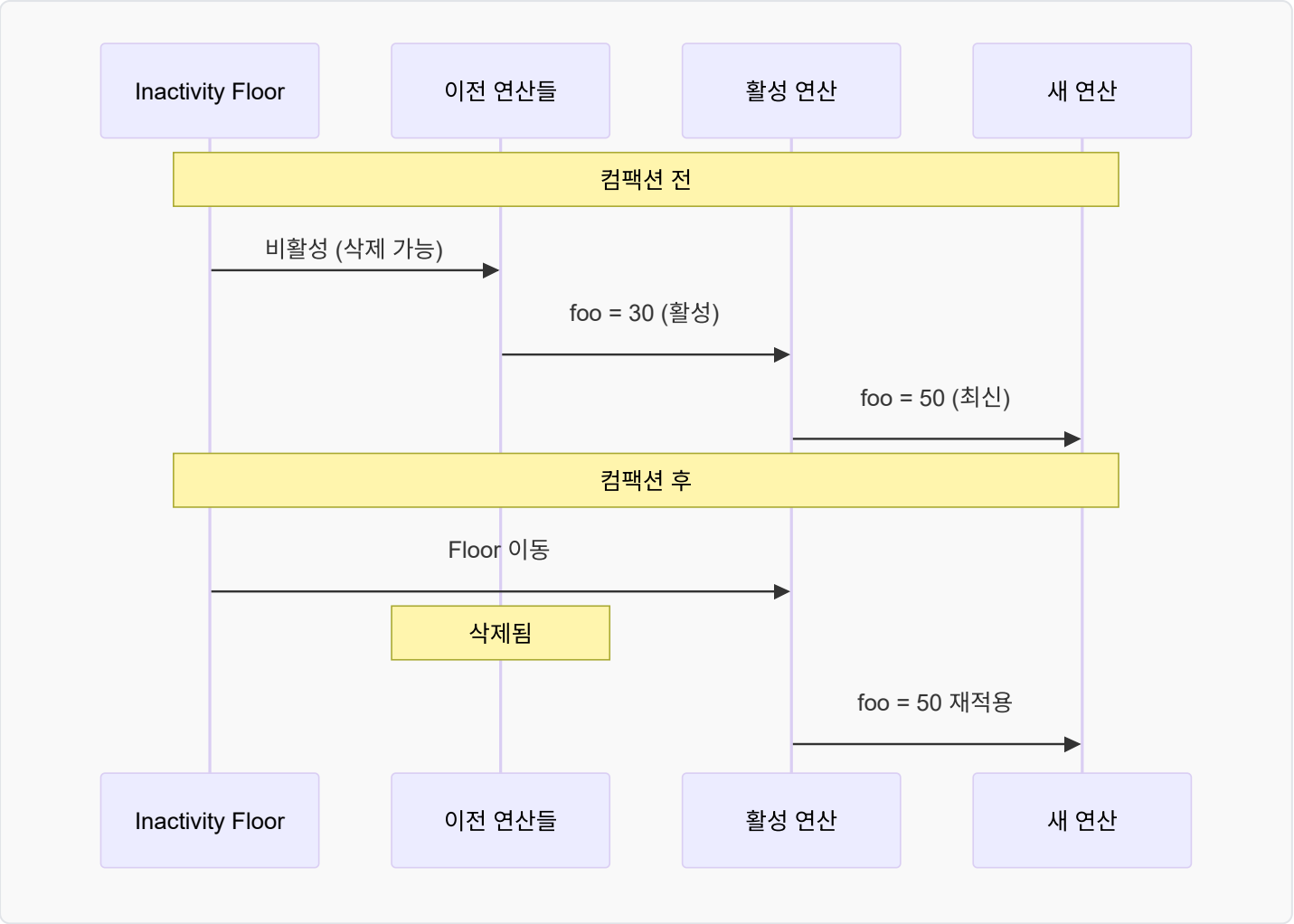
MMR 기반의 인증된 데이터베이스 제품군입니다.

5.2.1 adb::any

가장 단순한 형태로, **"어떤 시점에 키가 특정 값을 가졌음"**을 증명합니다.

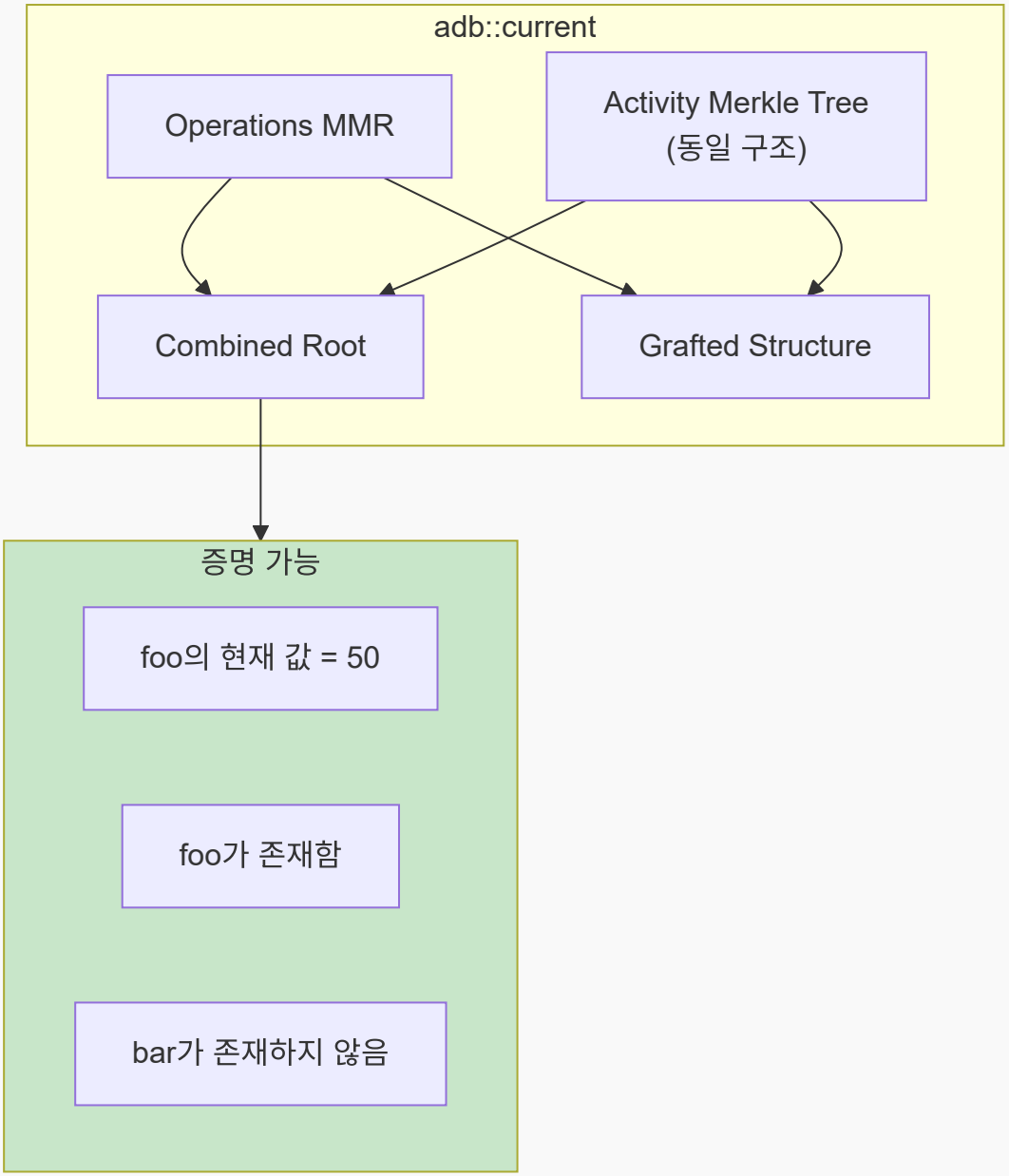


Compaction 프로세스:



5.2.2 adb::current

"키의 현재 값"을 증명할 수 있도록 확장된 버전입니다.



Grafting 기법:

```
 MMR 리프와 Activity 비트를 연결 (Grafting)

MMR:        [Op1] [Op2] [Op3] [Op4] [Op5]
Activity:   [ 0 ] [ 1 ] [ 0 ] [ 1 ] [ 1 ]
             ^inactive  ^inactive  ^active

current-value 증명 = any-value 증명 + activity 비트
```

5.3 QMDB 통합

Commonware는 LayerZero의 **QMDB (Quick Merkle Database)**와 협력하여 이를 라이브러리에 통합하고 있습니다.

> "Over the last year, we've developed a family of append-only, authenticated databases (ADB) inspired by LayerZero's novel Quick Merkle Database (QMDB). Today, we are excited to announce

> that this family of databases will now be known as QMDB."

QMDB 성능:

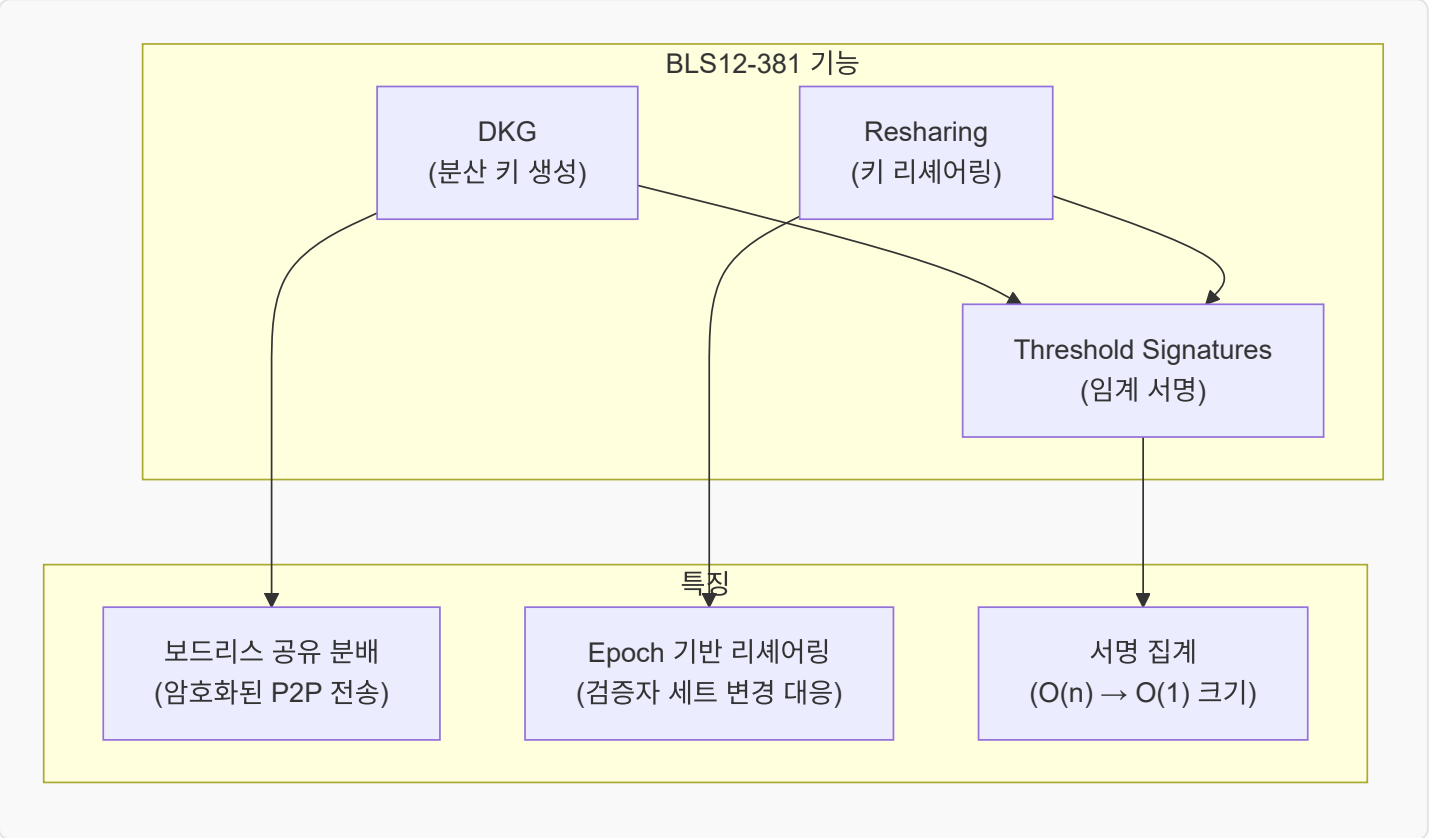
- 상태 읽기: 1 SSD read
- 상태 업데이트: O(1) SSD I/O
- Merkleization: 메모리 내 (0 SSD I/O)
- 메모리: 2.3 bytes/entry
- 처리량: 최대 2.28M 상태 업데이트/초

---

6. 암호화 프리미티브

6.1 cryptography::bls12381

BLS12-381 타원 곡선 기반 암호화 프리미티브입니다.



사용 예:

```
use commonware_cryptography::bls12381::{
    dkg::{Contributor, Phase},
    threshold::Signer,
};

// 1. DKG 실행 (n명의 참여자)
let contributor = Contributor::new(participants, threshold, my_index);

// Phase 1: 비밀 공유 생성 및 암호화 전송
```

```
let shares = contributor.generate_shares();
for (i, share) in shares.iter().enumerate() {
    send_encrypted(i, share);  // P2P 전송 (보드 없음)
}

// Phase 2: 공유 수신 및 검증
for share in received_shares {
    contributor.receive_share(share)?;
}

// Phase 3: 공개키 합의
let group_public_key = contributor.finalize()?;

// 2. 임계 서명
let signer = Signer::new(my_share, group_public_key);
let partial_sig = signer.sign(message);

// 3. 서명 조합 (임의의 t개 부분 서명)
let threshold_sig = combine(partial_sigs)?;  // 96 bytes
```

6.2 지원 스킴 비교

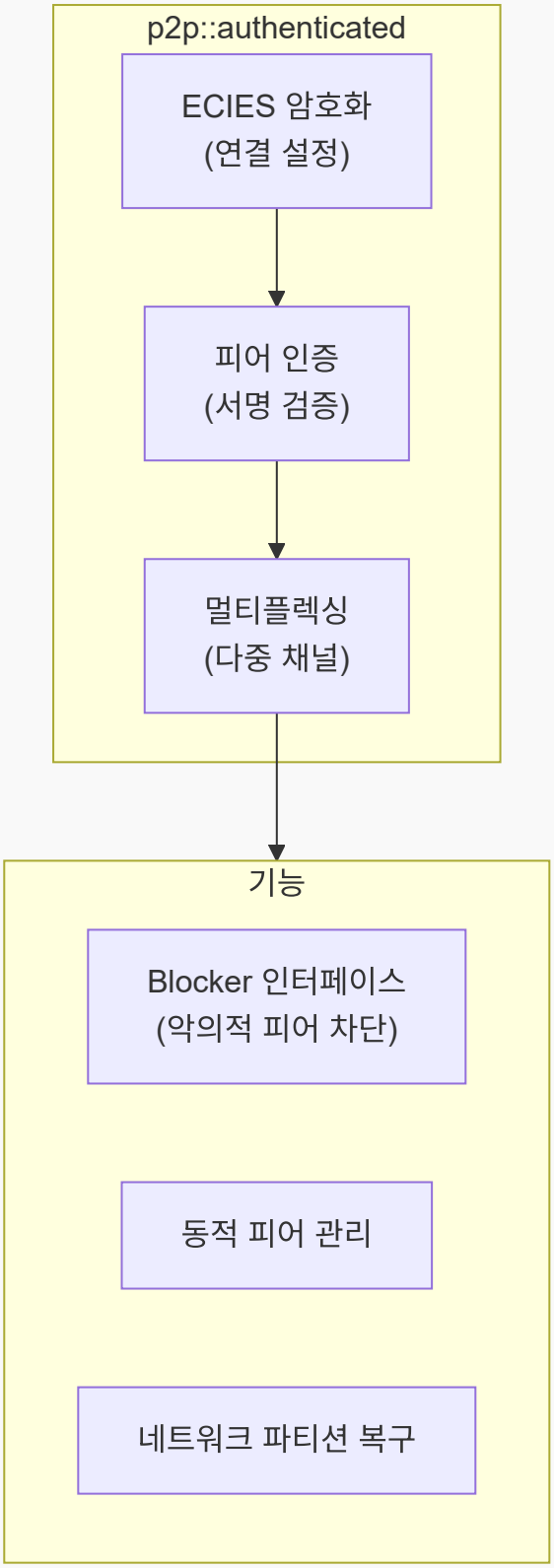
스킴	곡선	특징	사용 사례
ed25519	Curve25519	빠른 서명/검증, 배치 검증	일반 서명
bls12381	BLS12-381	서명 집계, 임계 서명	합의, 크로스체인
secp256r1	NIST P-256	HSM 호환	엔터프라이즈

---

7. 네트워킹 프리미티브

7.1 p2p::authenticated

인증된 P2P 네트워킹 프리미티브입니다.



7.2 p2p::simulated

결정론적 시뮬레이션을 위한 가상 네트워크입니다.

```
use commonware_p2p::simulated::{Network, Config};

let config = Config {
    latency: Duration::from_millis(50),
    packet_loss: 0.01,  // 1%
```

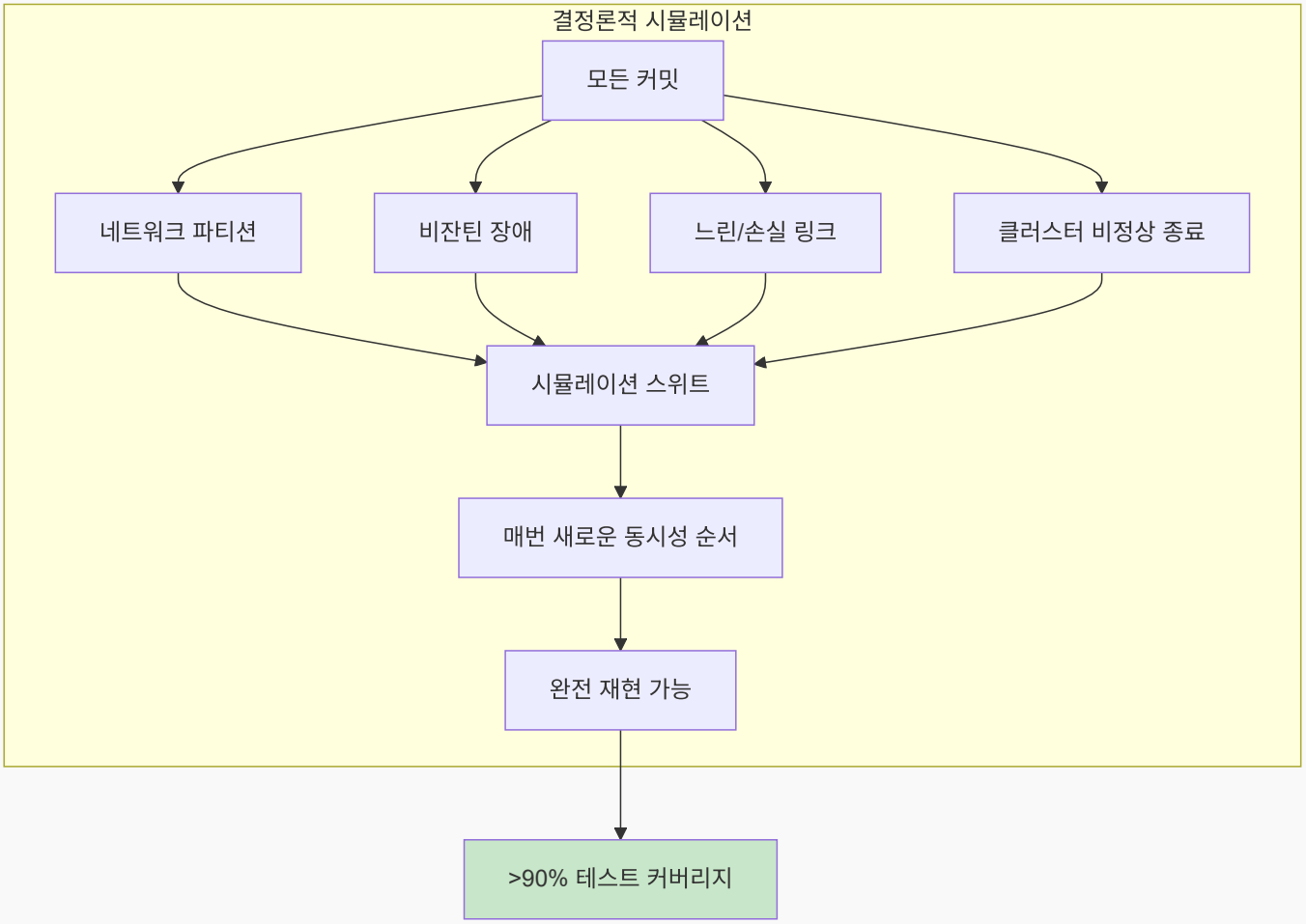
```
    partitions: vec![...],  // 네트워크 파티션
};

let network = Network::new(config);

// 결정론적 시뮬레이션 실행
// - 동일한 시드 → 동일한 결과
// - 디버거에서 단계별 실행 가능
```

---

8. 런타임 및 테스트

8.1 결정론적 시뮬레이션

Commonware의 핵심 품질 보증 방식입니다.



특징:

1. **결정론적**: 동일한 시드 → 동일한 실행 순서
2. **재현 가능**: 버그를 디버거에서 단계별로 추적
3. **포괄적**: 네트워크 파티션, 비잔틴 장애, 링크 손실 등 시뮬레이션
4. **자동화**: 모든 커밋에서 실행

8.2 예제 프로젝트: Alto

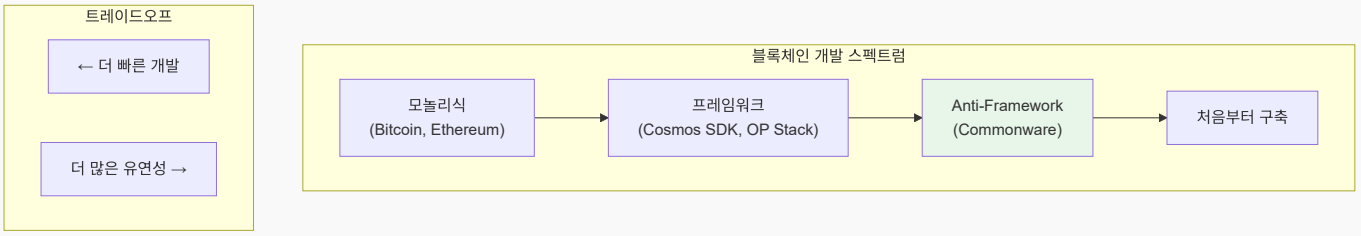
Commonware 프리미티브를 사용한 **미니멀 블록체인** 예제입니다.

```
Alto 성능:
- 블록 타임: ~200ms
- Finality: ~300ms
- CPU 사용량: 기존 대비 65% 감소

사용 프리미티브:
- consensus::threshold_simplex
- p2p::authenticated
- storage::mmr
- cryptography::bls12381
```

---

9. 결론 및 전망

9.1 Commonware의 위치



9.2 채택 현황

프로젝트	사용 프리미티브	상태
Tempo	simplex, broadcast, storage	테스트넷 (핵심 기여자)
Alto	threshold_simplex, p2p, mmr	벤치마크
(미정)	다양한 조합	개발 중

9.3 향후 로드맵

1. **QMDB 통합**: LayerZero와 협력하여 고성능 인증 데이터베이스 완성
2. **추가 프리미티브**: 더 많은 합의, 저장, 암호화 옵션
3. **MCP 서버**: LLM 통합을 통한 개발자 경험 향상
4. **문서화**: 포괄적인 가이드 및 예제

참고 자료

공식 자료

- **웹사이트**: https://commonware.xyz
- **GitHub**: https://github.com/commonwarexyz/monorepo
- **문서**: https://docs.rs/commonware-*
- **MCP 서버**: https://mcp.commonware.xyz

블로그 포스트

- Introducing Commonware (2024.10.30)
- Commonware: the Anti-Framework (2024.12.11)
- commonware-cryptography: Unlocking Seeds, Links, and Views (2025.01.06)
- Merkle Mountain Ranges for Performant Data Authentication (2025.02.13)
- Scaling Throughput with Ordered Broadcast (2025.03.18)
- The Simplest Database You Need (2025.04.09)
- Grafting Trees to Prove Current State (2025.07.09)
- Fast Block Dissemination with Immediate Guarantees (2025.11.04)

학술 자료

- Benjamin Y. Chan and Rafael Pass. “Simplex Consensus: A Simple and Fast Consensus Protocol”. TCC 2023.
- QMDB: Quick Merkle Database (arXiv:2501.05262)
- Merkle Mountain Ranges are Optimal (ePrint:2025/234)